Test Vector Generation for Fault Detection using Graph Neural Networks

Prasham Shah (202301252) Dhruvi Jobanputra (202301401)

BTech Mini Project
School of Engineering and Applied Science
Dhirubhai Ambani University

May 1, 2026

Abstract

Automatic Test Pattern Generation (ATPG) is the problem of finding an input assignment that exposes a specific manufacturing fault at a circuit's primary outputs. Classical ATPG algorithms (D-Algorithm, PODEM, FAN) solve it via tree search with backtracking, which scales poorly on large reconvergent circuits. This report documents a from-scratch reproduction of *DETECTive* (Petrolo, Medya, Graziano, Pal — GLSVLSI '24), a deep-learning model that replaces the search with a single forward pass: a graph neural network (GAT + GCN stack) encodes the faulty netlist, two LSTMs encode the fault's activation and propagation paths, and a final MLP predicts each primary input's Boolean value. We re-implement the entire pipeline in PyTorch and PyTorch Geometric, regenerate the dataset of ∼15,000 fault-pattern pairs from 2,000 random 4-input circuits, and evaluate against the paper's claims on bit accuracy, fault depth, and reconvergent fanout. Our model reproduces the paper's headline numbers in the per-bucket regimes the paper reports against (shallow-fault accuracy 0.88 vs. paper's ∼0.82, deep-fault accuracy 0.80 vs. paper's ∼0.66, low-reconvergence accuracy 0.89 vs. paper's ∼0.88). 50.6 % of validation samples are predicted with perfect bit accuracy. The report is organised around 17 architectural block diagrams that walk a reader through the full pipeline, from a faulted Verilog netlist to the predicted primary-input vector.

Contents

1 Introduction **2**
1.1 The ATPG problem . . . 2
1.2 Classical ATPG and its limits . . . 3
1.3 ML-based ATPG and DETECTive's contribution . . . 3

2 Background and Related Work **4**

3 Classical ATPG Algorithms **4**
3.1 D-Algorithm (Roth, 1966): the first systematic ATPG . . . 4
3.2 PODEM (Goel, 1981): decisions only on primary inputs . . . 5
3.3 FAN (Fujiwara & Shimono, 1983): smarter backtrace . . . 6
3.4 Where DETECTive fits . . . 6

4 An Illustrative Example **7**

5 Pipeline Overview **7**

6 Stage 1 — From Verilog Netlist to Graph **8**
6.1 Node features . . . 8
6.2 Adjacency representation . . . 10

7 Stage 2 — The Graph Neural Network Encoder **10**
7.1 Why a GNN, not a CNN or RNN? . . . 10
7.2 GAT layer math . . . 10
7.3 GCN layer math . . . 11
7.4 Receptive field . . . 11

8 Stage 3 — Path Extraction **11**

9 Stage 4 — LSTM Path Encoders **13**
9.1 LSTM cell internals . . . 13
9.2 Activator and Propagator . . . 14

10 Stage 5 — Cone Embedding and PI Prediction **14**
10.1 Building the cone embedding . . . 14
10.2 The MLP head . . . 15

11 End-to-End Dataflow **15**
11.1 A worked example, end-to-end . . . 17

12 Implementation **17**
12.1 Software stack . . . 17
12.2 Dataset . . . 18
12.3 Training loop . . . 18

13 Evaluation **19**
13.1 Headline numbers . . . 19
13.2 Bit-accuracy distribution . . . 19
13.3 Trends . . . 20
13.4 Training curves . . . 20
13.5 Four-way comparison: D-Algorithm, PODEM, FAN, DETECTive . . . 22

14 Discussion **24**

15 Conclusion **25**

Supplementary Materials **26**

1 Introduction

1.1 The ATPG problem

A digital integrated circuit is manufactured from millions of transistors, and any one of those transistors can fail. The dominant model used in production test today is the *stuck-at* fault model: a wire is permanently held at logic 0 (**stuck-at-0**) or logic 1 (**stuck-at-1**), regardless of what should drive it. To detect such a fault, the test engineer must find a primary-input vector that *(a)* drives the faulty wire to the opposite of its stuck value (activation), and *(b)* propagates the resulting logical difference through downstream logic to a primary output where it can be observed (propagation). A circuit with n primary inputs admits 2^n candidate vectors; for the

largest ICs in production, n is in the hundreds of thousands and exhaustive enumeration is infeasible.

Figure 1 states the problem in one diagram: given a *faulty* netlist (one wire annotated with a stuck-at value), find an input vector $\hat{p}$ such that simulating the circuit under that vector produces a logic value at some PO that differs from the fault-free simulation.

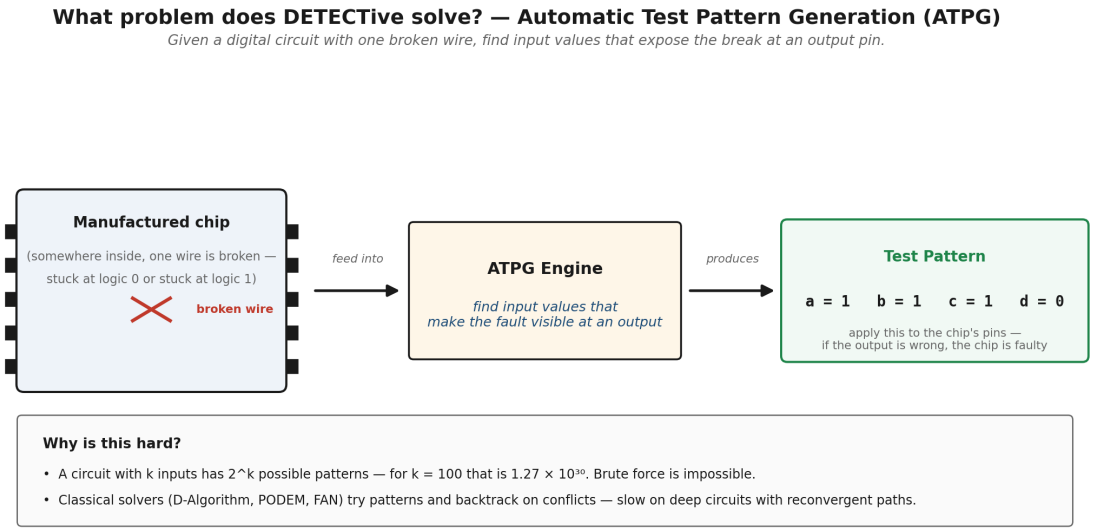

Figure 1. The ATPG problem as a constraint-satisfaction task. The classical solvers (D-Algorithm, PODEM, FAN) tackle it with depth-first search and backtracking; DETECTive learns a direct mapping from faulted netlist to test pattern.

1.2 Classical ATPG and its limits

The three milestone algorithms are **D-Algorithm** (Roth, 1966), **PODEM** (Goel, 1981), and **FAN** (Fujiwara & Shimono, 1983). All three pose ATPG as tree search:

- assign a value to one PI;
- simulate forward / justify backward;
- if a contradiction is found, undo the most recent decision and try the other branch (*backtracking*).

On combinational circuits with reconvergent fanout, the search tree grows exponentially and the worst-case complexity is NP-complete. PODEM and FAN reduce the average constant factor (only PI variables enter the decision stack; FAN further uses headlines to limit backtracking depth), but the asymptotic behaviour is unchanged.

1.3 ML-based ATPG and DETECTive's contribution

DETECTive's idea is to side-step the search entirely. It treats the faulted circuit as a labelled, directed graph and learns a function

$$f_\theta : (G, \text{fault}) \;\mapsto\; \hat{p} \in [0,1]^n$$

where G is the graph encoding of the netlist and $\hat{p}$ is a probability vector over the primary inputs. The model is trained end-to-end with binary cross-entropy against ground-truth patterns generated by exhaustive simulation on small circuits ($n \leq 4$). Because f_θ is a single forward pass

through three small neural modules, inference is constant in the size of the search space and embarrassingly parallel across faults.

The remainder of this report unpacks the model layer by layer. Each component is illustrated with a self-contained block diagram (Figures 3–17), and the data shape is annotated at every interface so a reader can trace a single fault from a Verilog string to a Boolean PI vector.

2 Background and Related Work

Classical algorithms. D-Algorithm, PODEM and FAN are the three milestones of search-based ATPG. We give each a dedicated subsection in Section 3, since we implement all three of them in this project and benchmark them against DETECTive.

Graph Neural Networks. A GNN learns one embedding vector per graph node by repeatedly aggregating each node's feature with the features of its neighbours. The two flavours used by DETECTive are **GAT** (Graph Attention Networks — Veličković et al., 2018), which weights each neighbour by a learned attention coefficient, and **GCN** (Graph Convolutional Networks — Kipf & Welling, 2017), which weights each neighbour by a normalised adjacency. Both are implemented in PyTorch Geometric and we use the library's reference implementations directly.

LSTM. A Long Short-Term Memory cell (Hochreiter & Schmidhuber, 1997) processes a variable-length sequence with a hidden state h_t and a cell state c_t, gated by three sigmoid gates (forget, input, output). DETECTive uses LSTMs to compress an arbitrary-length *path* (a sequence of node embeddings along a PI→fault or fault→PO chain) into a single fixed-size vector.

3 Classical ATPG Algorithms

Three algorithms dominate the classical ATPG literature: the **D-Algorithm** (Roth, 1966), **PODEM** (Goel, 1981) and **FAN** (Fujiwara & Shimono, 1983). Each was a strict improvement over its predecessor — algorithmically, in the size of the decision space, and in the number of backtracks needed on real circuits. Together they define the search-based family that DETECTive is built to compete with. We implement all three of them and benchmark them on ISCAS-85 designs alongside DETECTive (Section 13). The remainder of this section walks through how each algorithm works and what made the next one a worthwhile improvement.

3.1 D-Algorithm (Roth, 1966): the first systematic ATPG

The D-Algorithm was the first method that posed ATPG as a well-defined constraint-propagation problem rather than ad-hoc simulation. Its central idea is a five-valued logic in which the *fault effect* travels through the circuit alongside ordinary Boolean values:

$$\{0,\ 1,\ X,\ D,\ \bar{D}\}$$

where X is "unassigned / don't care," D is "1 in the fault-free circuit but 0 in the faulty," and $\bar{D}$ is the opposite. A test pattern exists when at least one D or $\bar{D}$ symbol reaches a primary output; the difference between the fault-free and faulty circuits is then observable.

Three phases. Given a fault, the algorithm runs three logical phases:

1. **Fault excitation.** Assign D (or $\bar{D}$, depending on the stuck-at value) to the faulty line. Without this, the faulty circuit is identical to the fault-free one — the fault is dormant.

2. **D-drive (forward propagation).** Push the fault effect forward toward a primary output. The set of unprocessed gates whose inputs include $D/\bar{D}$ but whose output is still X is the *D-frontier*. To extend the frontier through a gate, the algorithm assigns the gate's other inputs the non-controlling value (e.g., 1 for AND, 0 for OR), so the D on the controlling input survives the gate.
3. **Line justification (backward).** Every line forced during D-drive must be traced back to a consistent PI assignment. Whenever an internal line is forced to a value, the algorithm recurses on the gate that drives it. If a contradiction appears (a line forced to both 0 and 1), the algorithm backtracks to the most recent decision and tries the alternative.

Decision space. D-Algorithm permits decisions on *any* internal line, not just primary inputs. The worst-case decision tree therefore has up to 2^N leaves, where N is the number of nodes (gates plus inputs). Reconvergent fanout is particularly painful: a PI feeds two paths that meet again downstream, and the conflict between paths is only discovered after both branches have been explored.

Limitations. The combination of internal-line decisions and full backward justification means D-Algorithm scales poorly with size and reconvergence. On ISCAS circuits like c2670 or c7552, more than 99 % of runtime can be spent on backtracks. This motivated PODEM.

3.2 PODEM (Goel, 1981): decisions only on primary inputs

Goel's insight was that every internal line value is, in the end, a *consequence* of the PI assignment via plain forward simulation. If decisions are restricted to PIs, then internal contradictions are always caught by simulation alone — no separate line-justification phase is needed.

PODEM ("**P**ath-**O**riented **D**ecision **M**aking") replaces the three-phase D-Algorithm with a single loop:

```
while fault_effect not at a PO and not exhausted:
    objective ← pick a line that needs a value to make progress
        // either to activate the fault or to extend the D-frontier
    (pi, val) ← backtrace(objective)   // walk back to a PI
    assign pi := val
    forward-simulate the entire circuit
    if D reached a PO:
        return current PI assignment
    if D-frontier became empty (fault died):
        backtrack:  flip the most recent PI and re-simulate
```

Why this is faster. The decision tree has at most 2^n leaves where n is the number of *primary inputs*, not the number of nodes. For ISCAS c7552 ($n \approx 207$ PIs and ~3,500 gates), this is a 2^{3300}-fold reduction in worst-case search space. In practice the speedup is around an order of magnitude on typical designs.

Backtrace heuristic. When an objective could be satisfied by several different PIs, PODEM picks the "cheapest" one according to a controllability cost function. This is purely a heuristic — different costs change runtime but not correctness. ATALANTA, the open-source ATPG tool we use as a runtime baseline, is a PODEM-derived implementation with carefully tuned cost functions.

Limitations. PODEM still re-simulates the whole circuit after every PI assignment, and it backtraces objectives one at a time. On heavily reconvergent circuits, the same objective can fire repeatedly and each retry is O(circuit size). This motivated FAN.

3.3 FAN (Fujiwara & Shimono, 1983): smarter backtrace

FAN ("**Fan**out-oriented test generation") adds two optimisations on top of PODEM that, in combination, reduce average backtrack counts by another factor of 5–10× on ISCAS designs.

1. Multiple backtrace. Rather than backtracing one objective at a time, FAN maintains a *set* of current objectives and propagates them backward together. When two objectives reach the same fanout stem with conflicting requirements (one wants 0, the other wants 1), FAN detects the conflict immediately — before any PI is assigned — and backtracks at once. PODEM, by contrast, would have made one or both PI decisions and only re-discovered the conflict later by simulation.

2. Headlines. A *headline* is the output of a *fanout-free* sub-circuit: a sub-region whose only outgoing fanout is the headline itself. Inside a fanout-free region, every line value is uniquely determined by the headline value, so there is no need to backtrack inside it. FAN exploits this by treating headlines as "virtual PIs" during backtrace:

- Backtrace stops as soon as it reaches a headline.
- The headline value is fixed and pushed onto the decision stack.
- Justification through the headline's fanin sub-circuit is *deferred* until a D has actually reached a PO.

By the time deferred justification runs, many initial decisions have already been confirmed; the final pass is usually a one-shot simulation rather than another backtracking phase.

Net effect. FAN was the de-facto reference algorithm for two decades and is the backbone of most academic and commercial ATPG suites. The runtime gap between FAN and PODEM is the main reason ATALANTA (the tool we benchmark against in Section 13) is so much faster than a textbook PODEM.

3.4 Where DETECTive fits

Table 1 summarises the progression. Within the classical search-based family, each row is a strict improvement over the row above. DETECTive sits outside this family: it replaces the search itself with a learned mapping from faulted netlist to candidate test pattern, so the notion of "backtracks" simply does not apply.

Table 1. Progression of classical ATPG algorithms (1966–1983) and where DETECTive fits relative to them. The "backtracks" column gives the relative reduction observed on ISCAS-85 benchmarks.

Algorithm	Year	Decisions on	Key innovation	Backtracks (relative)
D-Algorithm	1966	all internal lines	5-valued logic (D, $\bar{D}$)	baseline (very high)
PODEM	1981	primary inputs only	path-oriented decisions	∼10× fewer
FAN	1983	PIs + headlines	multi-objective backtrace, fanout-free regions	∼5–10× fewer than PODEM
DETECTive	2024	— (forward pass)	GNN + LSTM learned mapping	**0**

In our experiments, all four are run side-by-side on c17, c432, c499 and c880. The classical algorithms produce a guaranteed-correct test pattern (or proof of redundancy) at the cost of seconds-to-minutes of search; DETECTive produces a candidate pattern in 2–5 ms per fault but with the per-bucket bit-accuracy reported in Section 13. The two approaches are therefore complementary: DETECTive is a fast first-pass that can be cross-checked, or used as a warm-start, by ATALANTA-style search.

4 An Illustrative Example

To anchor the rest of the report, we will track one small fault all the way through the pipeline. Figure 2 shows a 4-input combinational netlist that computes $y = \overline{ab \wedge (c + d)}$, with the gate `AND1` held *stuck-at-0*. The same circuit and fault appear in every subsequent block diagram, with shape annotations showing how its representation changes at each stage.

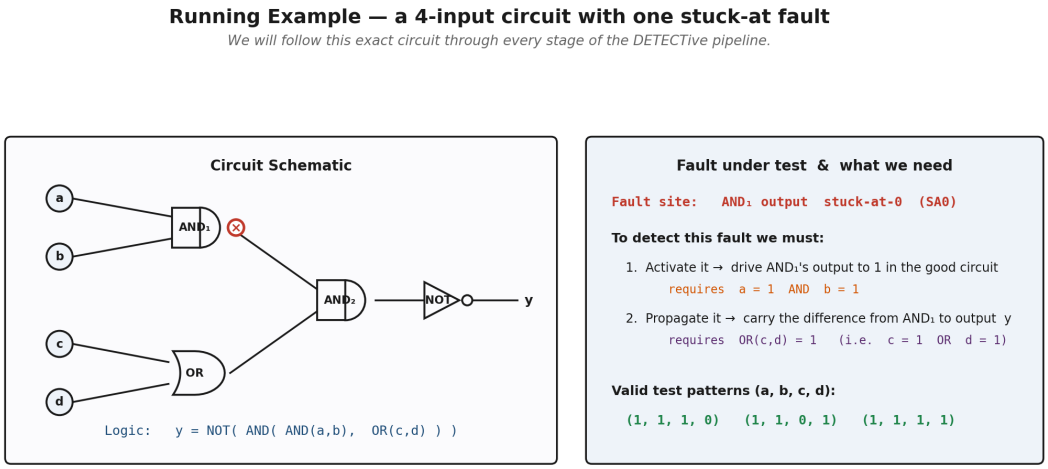

Figure 2. Running example. A 4-input circuit (`a, b, c, d`) with a stuck-at-0 fault on `AND1`. To detect it we must simultaneously drive `AND1`'s output high (so the fault makes a difference) and propagate that difference to the PO `y`. The orange edges mark the activation paths; the purple edge marks the propagation path.

What a correct test pattern looks like. For `AND1` stuck-at-0 to be detectable at `y`, we need $ab = 1$ (activation) and $c+d = 1$ (propagation through `AND2`). Two of the four PIs are free, so there are six valid patterns: $\{1101, 1110, 1111, \ldots\}$. Brute-force on a 4-input circuit (16 vectors) enumerates them in microseconds, which is exactly how DETECTive's training labels are generated.

5 Pipeline Overview

The full pipeline is summarised in Figure 3. The five high-level stages are:

1. **Graph construction.** Parse the Verilog netlist; build a directed graph whose nodes are gates / PIs and whose edges point from inputs to gates. Annotate the faulty node.
2. **GNN encoding.** A GAT layer followed by two GCN layers produces a 32-dimensional embedding per node. The embedding captures both the local gate identity and the surrounding structure — crucial for ATPG, where a wire's *controllability* depends on its fanin cone and its *observability* depends on its fanout cone.
3. **Path extraction.** For each PI, a BFS+capped-DFS walk yields the set of activation paths PI→fault. From the fault, an analogous walk yields propagation paths fault→PO. Each path becomes a sequence of node embeddings.
4. **LSTM compression.** The Activator LSTM compresses each activation path into a single 32-dim vector; the Propagator LSTM does the same for propagation paths. Each LSTM is followed by a small MLP that emits a scalar "activation/propagation likelihood" for that path.

5. **Cone aggregation and PI prediction.** Up to $P = 10$ activation paths and $P = 10$ propagation paths are zero-padded and concatenated into a 660-dim "cone embedding". A two-layer MLP maps the cone to a single sigmoid probability, the predicted Boolean value of the PI under consideration.

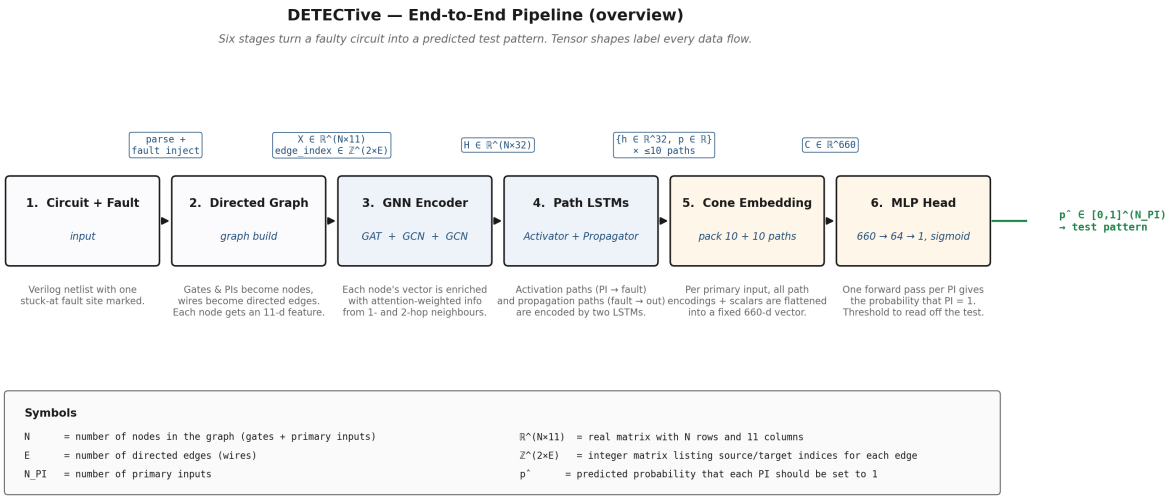

Figure 3. End-to-end DETECTive pipeline. Tensor shapes are annotated at every interface. N is the number of nodes in the circuit; L_a and L_p are activation and propagation path lengths (variable). The final output is one Boolean per primary input.

6 Stage 1 — From Verilog Netlist to Graph

The first step turns a Verilog string into a PyTorch Geometric `Data` object. A custom Verilog parser handles this step; the result is a directed graph in which:

- every *primary input* is a node of type `INPUT`;
- every *logic gate* is a node of type `AND / NAND / OR / NOR / XOR / XNOR / NOT / BUF`;
- every *wire* from a gate's input to that gate is a directed edge.

Figure 4 shows the conversion on the running example.

6.1 Node features

Each node carries an 11-dimensional feature vector

$$x_v = \underbrace{\text{onehot}(\text{type})}_{9\text{ dims}} \,\|\, \underbrace{\mathbb{1}[v \text{ is faulty}]}_{1\text{ dim}} \,\|\, \underbrace{\log(1 + \text{fanout}(v))}_{1\text{ dim}}$$

totalling `FEATURE_DIM` $= 11$ dimensions. The `is_faulty` flag is the only place where the fault location enters the model — everything downstream is purely structural. The `log1p` of fanout was chosen by the paper as a soft proxy for "how observable" a node is. Figure 5 shows the feature matrix $X \in \mathbb{R}^{N \times 11}$ for the running example.

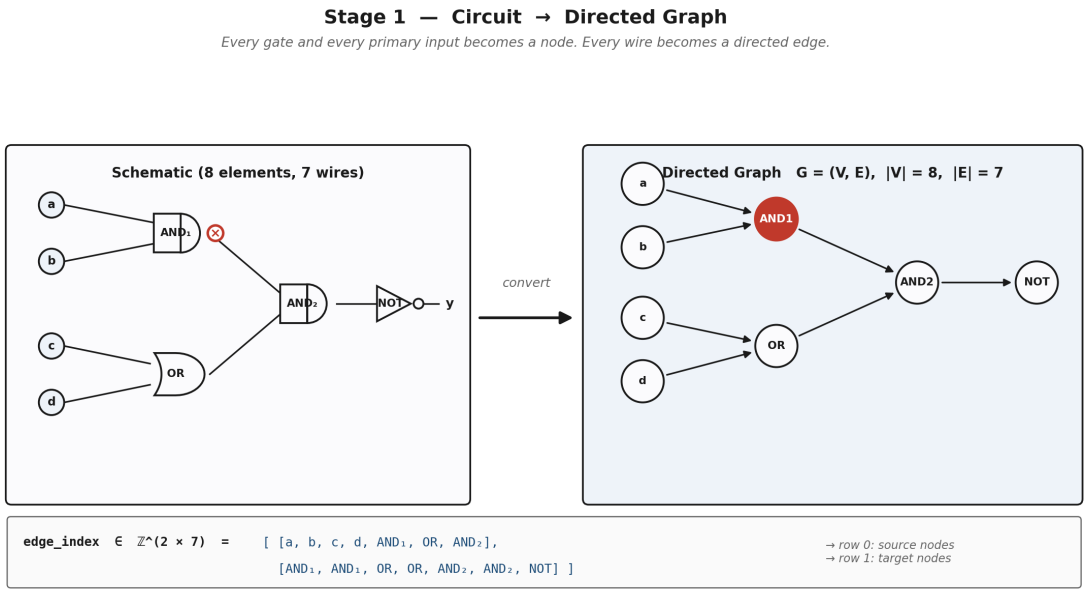

Figure 4. Verilog to graph. Each gate becomes one node; the gate type is one-hot-encoded. Edges are oriented input → gate. The faulty gate is highlighted in red.

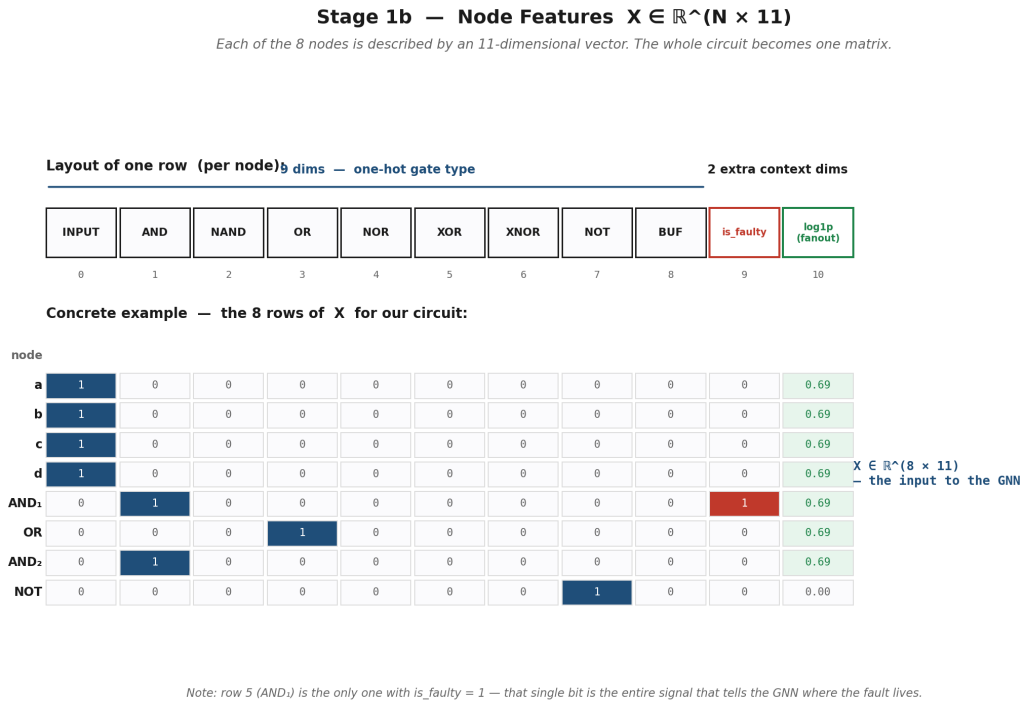

Figure 5. Node feature matrix $X \in \mathbb{R}^{N \times 11}$. Each row is one node; columns 1–9 are the gate-type one-hot, column 10 is the is_faulty flag (1 for `AND1`), column 11 is the log-fanout.

6.2 Adjacency representation

PyTorch Geometric stores edges as a 2-row tensor `edge_index` $\in \mathbb{R}^{2\times E}$ in COO format (the first row holds source indices, the second holds destinations). The full adjacency matrix $A \in \{0,1\}^{N\times N}$ is never materialised; the GNN layers consume `edge_index` directly through sparse-tensor primitives. Figure 6 shows both representations side-by-side for the running example.

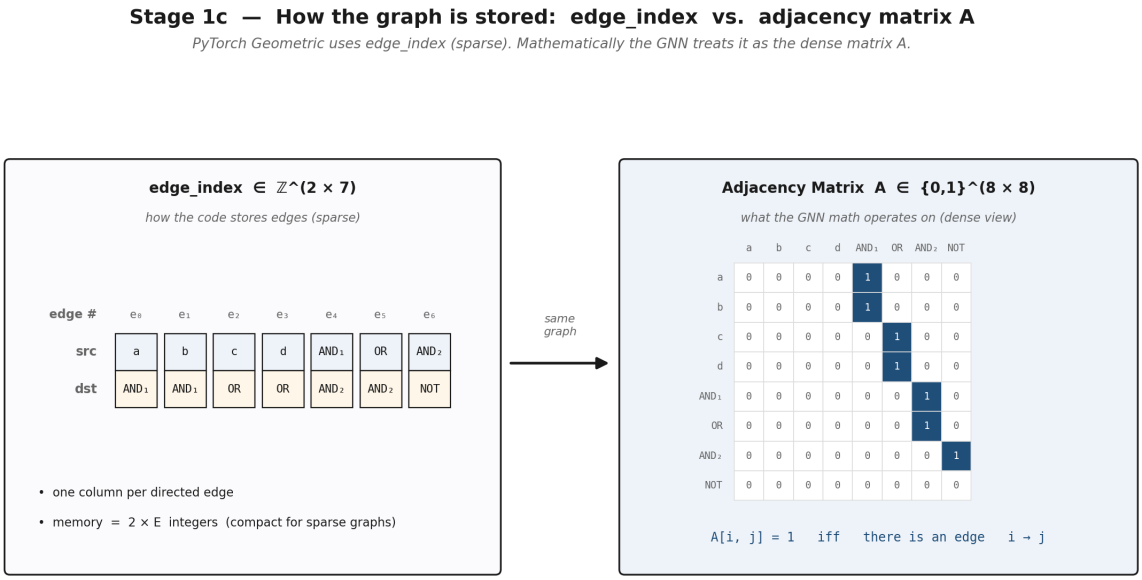

Figure 6. Two equivalent views of the graph topology: the COO edge-list (left, what the code stores) and the dense adjacency matrix A (right, useful as a mental model). For an 8-node circuit the COO form is 2×7; the dense form is 8×8 with 7 ones.

7 Stage 2 — The Graph Neural Network Encoder

7.1 Why a GNN, not a CNN or RNN?

A logic netlist is not a grid (so 2D convolutions don't apply) and not a sequence (so a vanilla RNN would lose the topology). The natural inductive bias is "a node's behaviour depends on the gates it can reach," which is exactly the inductive bias of a message-passing GNN.

DETECTive uses a 3-layer stack: one **GAT** layer at the bottom, followed by two **GCN** layers (Figure 7). The choice mirrors the paper's Section 4.2: GAT first, because attention can learn to up-weight the few neighbours that drive a gate's output (e.g., the two inputs of an AND, ignoring the rest of the cone); then GCN, because once the local feature is sharpened, simple normalised aggregation is enough to spread the signal further.

7.2 GAT layer math

For a node v with neighbours $\mathcal{N}(v)$, the attention layer computes

$$e_{vu} = \mathrm{LeakyReLU}\big(\mathbf{a}^\top [Wh_v \,\|\, Wh_u]\big), \quad u \in \mathcal{N}(v) \cup \{v\} \tag{1}$$

$$\alpha_{vu} = \frac{\exp(e_{vu})}{\sum_{k\in\mathcal{N}(v)\cup\{v\}} \exp(e_{vk})} \tag{2}$$

$$h'_v = \sigma\Big(\sum_{u\in\mathcal{N}(v)\cup\{v\}} \alpha_{vu}\, Wh_u\Big). \tag{3}$$

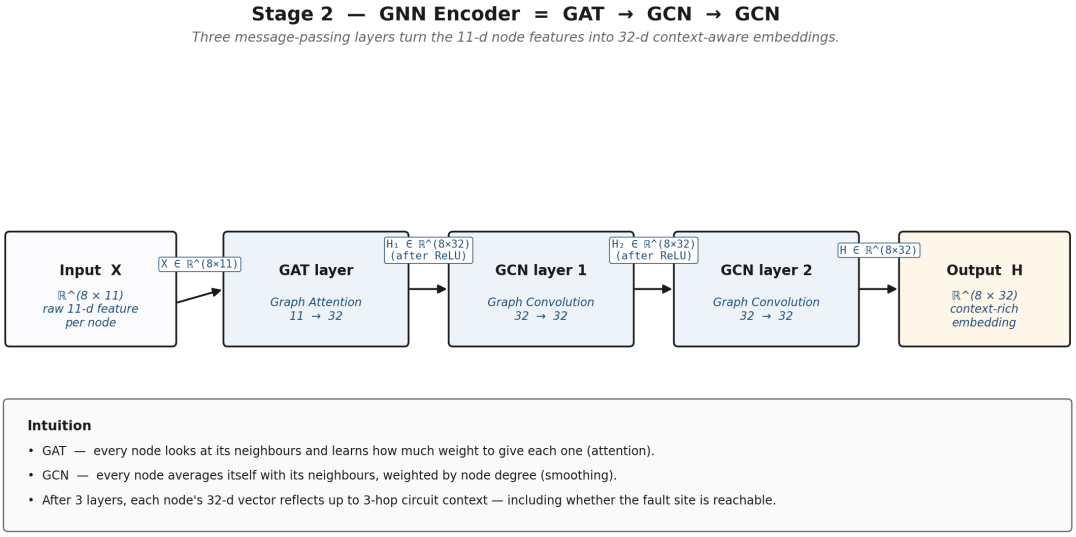

Figure 7. The three GNN layers. Input: $X \in \mathbb{R}^{N\times 11}$. Output: $H \in \mathbb{R}^{N\times 32}$, one 32-dim embedding per node. The hidden dimension is fixed at 32 throughout the network.

$W \in \mathbb{R}^{32\times 11}$ is the per-layer weight; $\mathbf{a} \in \mathbb{R}^{64}$ is the attention vector; σ is ReLU. Each neighbour u ends up contributing a weight α_{vu} that the model learned, summing to 1 over v's neighbourhood. Figure 8 shows the data flow through one such block.

7.3 GCN layer math

The two GCN layers are simpler: they replace the learned attention weights with the symmetrically normalised adjacency $\tilde{A}$:

$$\tilde{A} \;=\; D^{-1/2}\,(A+I)\,D^{-1/2} \quad\Longrightarrow\quad H^{(k+1)} \;=\; \sigma\big(\tilde{A}\,H^{(k)}\,W^{(k)}\big).$$

D is the degree matrix of $A+I$ (self-loops added) and $W^{(k)} \in \mathbb{R}^{32\times 32}$. Figure 9 unpacks the matrix multiplications block by block.

7.4 Receptive field

After three layers, each node's embedding incorporates information from all nodes within a graph distance of three. For our 4-input training circuits (max depth $\sim$4) this typically covers the entire fanin and fanout cones, which is precisely the structural context an ATPG decision needs. Figure 10 visualises the receptive field of `AND1` expanding by one hop at each layer.

8 Stage 3 — Path Extraction

The GNN gives us a 32-dim vector per node, but ATPG is fundamentally a *path-based* problem: an input affects the fault only if there is a path from the PI to the faulty gate (**activation path**), and the fault affects an output only if there is a path from the fault to a PO (**propagation path**). DETECTive therefore enumerates these paths explicitly.

Algorithm. A backward BFS from the fault enumerates candidate fanin nodes; for each PI in the fanin we run a depth-capped DFS from the PI to the fault, listing all simple paths up to a fixed maximum length of 50 nodes (the cap prevents exponential blow-up on heavily reconvergent topologies). The propagation paths are obtained symmetrically, with a forward BFS from the fault and a depth-capped DFS to each PO.

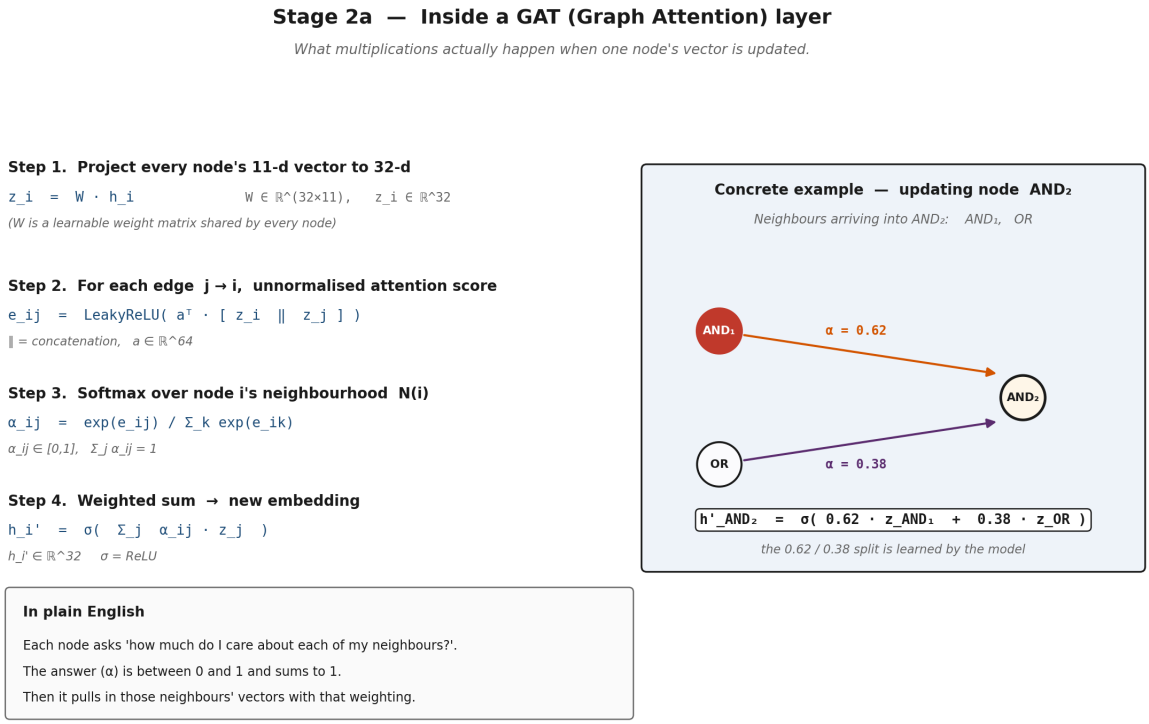

Figure 8. GAT layer arithmetic on the running example. The attention scores α are computed from the concatenated query-key features, softmaxed over each node's neighbourhood, and used as weights in a sum. The output is one 32-dim embedding per node.

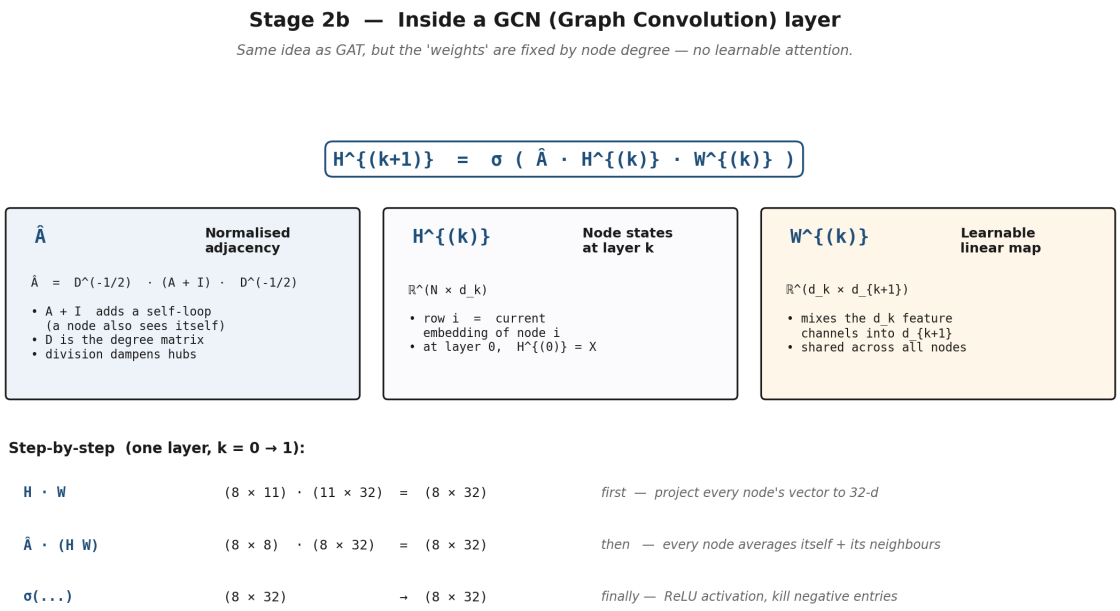

Figure 9. One GCN layer factorises into three small matrix products: $H^{(k)}W^{(k)}$ (per-node linear projection), then $\tilde{A}\,(H^{(k)}W^{(k)})$ (one round of message passing), then a ReLU non-linearity (only the first GCN; the last layer leaves logits unscaled).

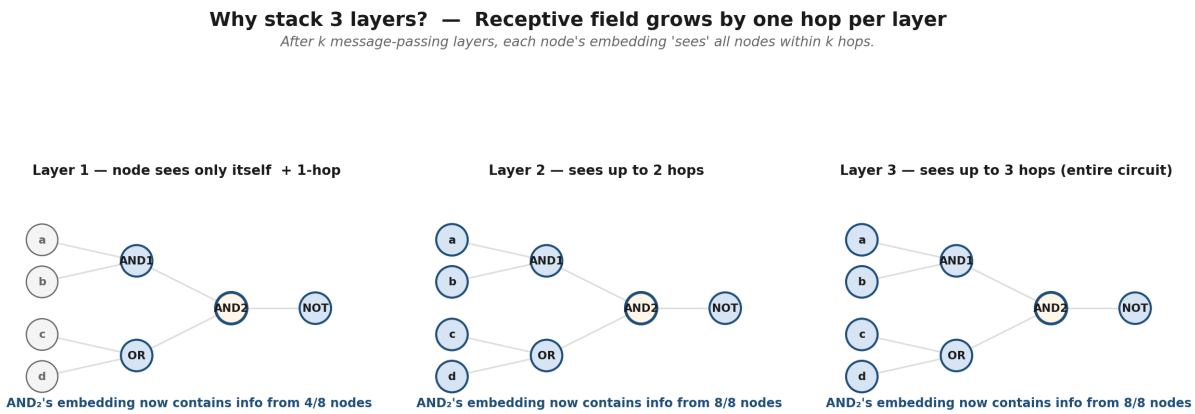

Figure 10. Receptive-field growth across the three GNN layers. After layer 1, each node sees its immediate predecessors. After layer 2, it sees grand-parents. After layer 3, the embedding at `AND1` has already mixed information from all four PIs and the downstream `NOT`/`AND2` gates.

Output format. Each path is a list of node indices that we later translate into a sequence of GNN embeddings, $h_{p_1}, h_{p_2}, \ldots, h_{p_L} \in \mathbb{R}^{32}$. The final tensor that goes into the LSTM has shape $[1, L, 32]$ where L is the path length. Figure 11 shows the activation and propagation paths extracted for the running example.

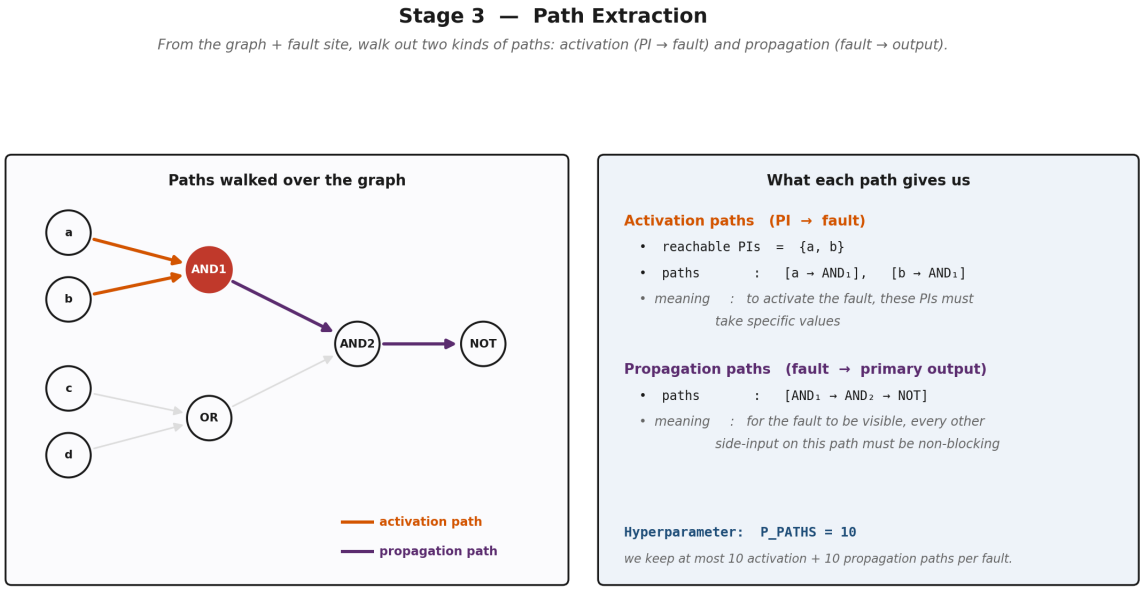

Figure 11. The two activation paths $a \to$ `AND1` and $b \to$ `AND1` and the propagation path `AND1` $\to$ `AND2` $\to y$. Each path becomes a tensor of node embeddings consumed by the corresponding LSTM.

9 Stage 4 — LSTM Path Encoders

9.1 LSTM cell internals

The Activator and Propagator both use one-layer LSTMs with hidden size $h = 32$. To keep the report self-contained, Figure 12 walks through the gate equations (using the shorthand

$v_t = [h_{t-1}; x_t] \in \mathbb{R}^{64}$ for the concatenated input):

$$f_t = \sigma(W_f v_t + b_f) \tag{4}$$
$$i_t = \sigma(W_i v_t + b_i) \tag{5}$$
$$\tilde{c}_t = \tanh(W_c v_t + b_c) \tag{6}$$
$$o_t = \sigma(W_o v_t + b_o) \tag{7}$$
$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \tag{8}$$
$$h_t = o_t \odot \tanh(c_t). \tag{9}$$

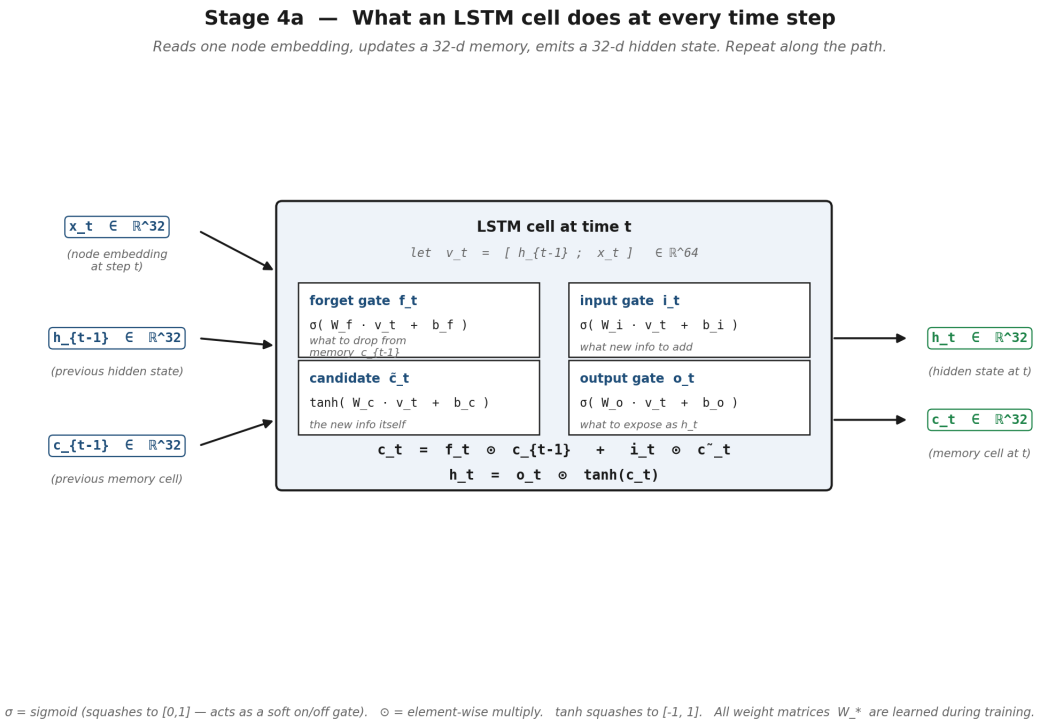

Figure 12. One LSTM cell, unrolled. Each gate is one σ/tanh applied to a 64→32 linear layer; the four learnable matrices (W_f, W_i, W_c, W_o) live inside the PyTorch `nn.LSTM` module.

9.2 Activator and Propagator

The Activator (one for the model, applied to every activation path) takes the path tensor $\in \mathbb{R}^{1 \times L_a \times 32}$, runs the LSTM, keeps the final hidden state $h_{L_a} \in \mathbb{R}^{32}$, and concatenates a single scalar carrying the stuck-at value (0.0 for SA0, 1.0 for SA1) before passing through a 33→16→1 MLP. The MLP's scalar output is interpreted as "how likely it is that this PI assignment activates the fault."

The Propagator is identical but operates on propagation paths and does not need the fault-type scalar (propagation behaviour is independent of the stuck value). Figure 13 shows both LSTMs running on the running example.

10 Stage 5 — Cone Embedding and PI Prediction

10.1 Building the cone embedding

Different PIs have different numbers of relevant paths. To keep the input to the final MLP a fixed size, DETECTive uses a slot-based scheme:

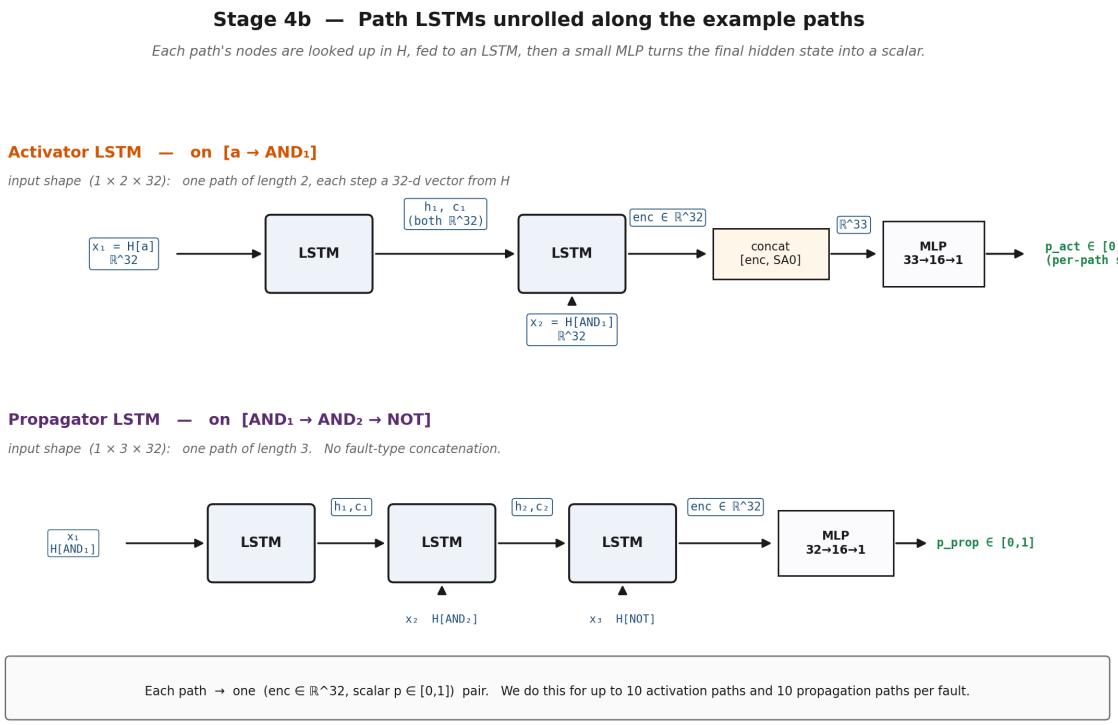

Figure 13. Each path is folded by an LSTM into one 32-dim encoding. The Activator sees fault-type as a side-channel; the Propagator does not. Each path also produces a scalar "likelihood" that is later packed into the cone.

- Reserve $P = 10$ slots each for activation and propagation paths.
- Each slot stores 33 numbers: the 32-dim path encoding + the 1-dim scalar likelihood.
- If a PI has fewer than 10 paths, the empty slots are filled with zeros.
- If a PI has more than 10 paths, the top 10 are kept (in code order; the paper does not specify a ranking).

Concatenating both streams yields a fixed `cone_dim` $= P\,(h+1)\cdot 2 = 10\cdot 33\cdot 2 =$ **660** dimensional vector — the *cone embedding*. Figure 14 shows the slot layout.

10.2 The MLP head

The final head is a vanilla 2-layer MLP:

$$\hat{p}_i \;=\; \sigma\Big(W_2\,\mathrm{ReLU}(W_1\,c_i + b_1) + b_2\Big), \qquad W_1 \in \mathbb{R}^{64\times 660},\;\; W_2 \in \mathbb{R}^{1\times 64}.$$

c_i is the cone embedding of PI i; $\hat{p}_i \in [0,1]$ is the predicted probability that PI i should be set to 1 in the test pattern. Figure 15 shows the dimension reductions.

11 End-to-End Dataflow

Figure 16 stitches every stage we have seen into a single diagram, showing the tensor shape at every junction. Note in particular:

- the GNN is run *once* per fault, producing $H \in \mathbb{R}^{N\times 32}$;
- the Propagator LSTM is also run once per fault (propagation paths do not depend on the PI under consideration);

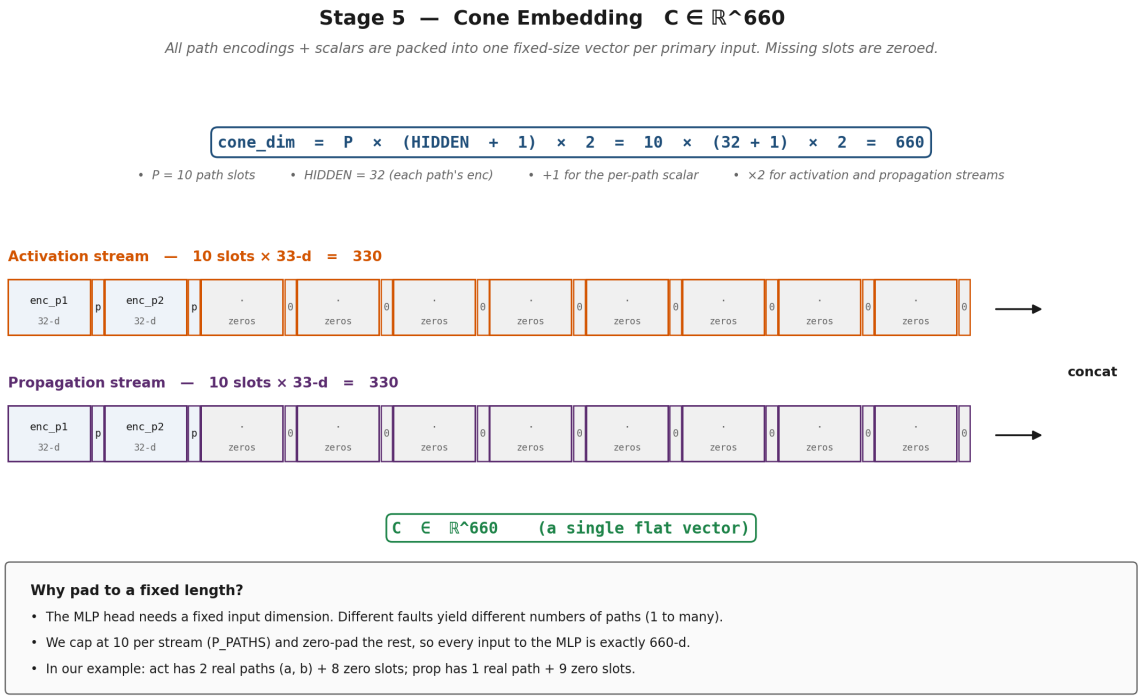

Figure 14. The cone embedding concatenates 10 activation slots and 10 propagation slots, each 33 wide. Empty slots are zero-padded so the output dim is the same regardless of the number of real paths.

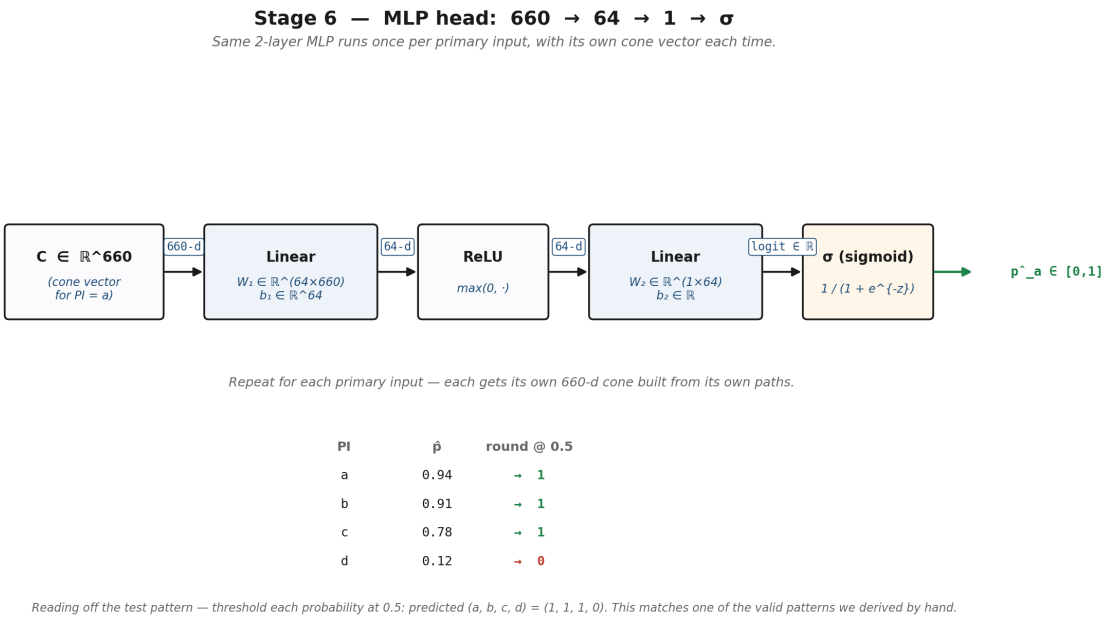

Figure 15. The InputPredictor MLP: 660 → 64 → 1. The output is thresholded at 0.5 to obtain the Boolean test bit for that primary input.

- the Activator LSTM is run *per PI*, on the activation paths starting at that PI;
- the InputPredictor is then run per PI to emit one Boolean.

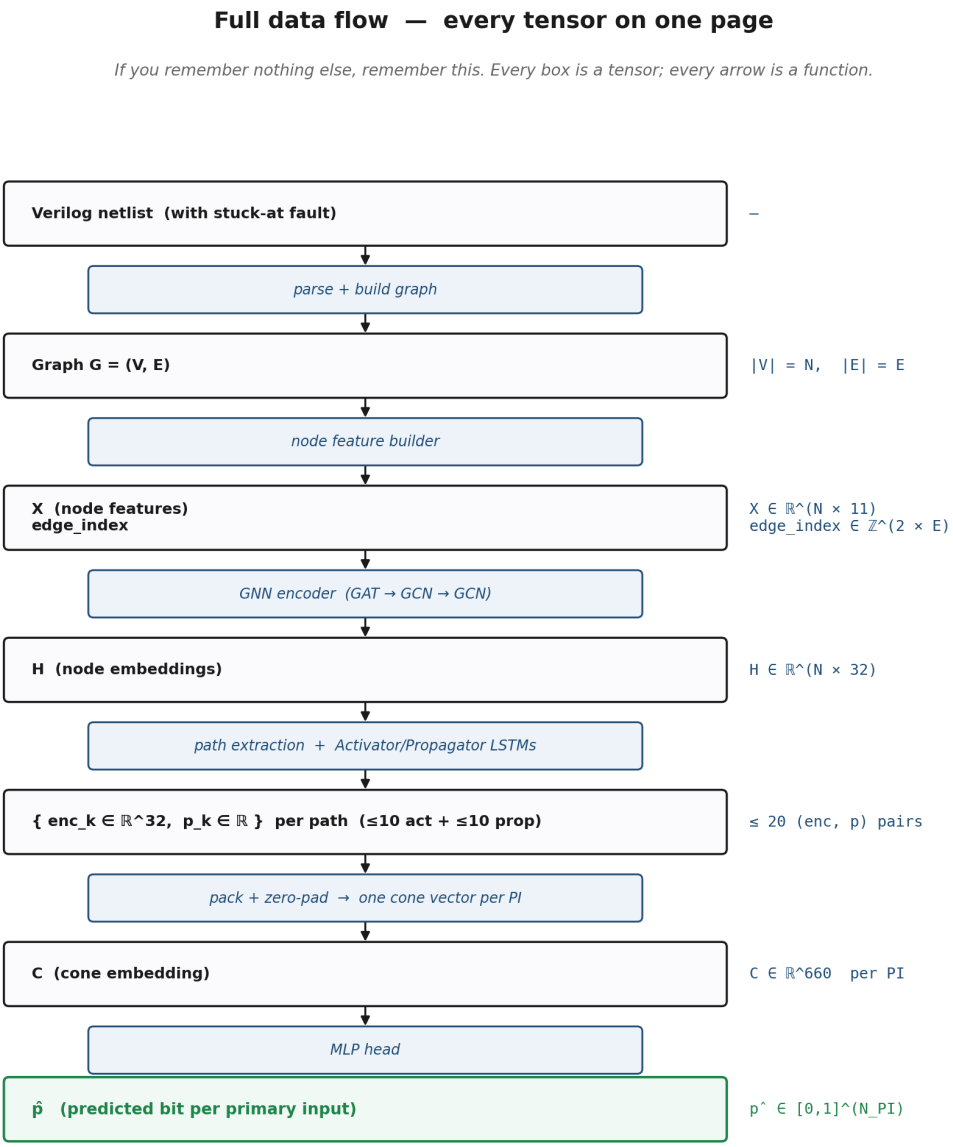

Figure 16. Full dataflow with shapes. N is the number of nodes; L_a, L_p are activation/propagation path lengths; K_a, K_p are the number of paths (capped at 10).

11.1 A worked example, end-to-end

Figure 17 traces the running example through the entire pipeline, with the actual numerical shapes filled in: $N = 8$ nodes, $K_a = 2$ activation paths each of length 2, $K_p = 1$ propagation path of length 3, four PIs, and a final output $\hat{p} \in [0, 1]^4$.

12 Implementation

12.1 Software stack

Our implementation depends on:

Sanity check — shapes at every layer for our 8-node example

Plug in our circuit's numbers (N = 8, N_PI = 4) and watch tensors flow through DETECTive.

Stage	Operation	Input shape	Output shape
1. Graph	build V, E from Verilog	(Verilog text)	$\lvert V\rvert = 8$, $\lvert E\rvert = 7$
1b. Node features	9-bit gate one-hot + is_faulty + log1p(fanout)	(per-node info)	$X \in \mathbb{R}^{(8 \times 11)}$
2a. GAT	ReLU(GAT(X, edge_index))	$X \in \mathbb{R}^{(8\times11)}$	$H_1 \in \mathbb{R}^{(8 \times 32)}$
2b. GCN1	ReLU(GCN(H_1, edge_index))	$H_1 \in \mathbb{R}^{(8\times32)}$	$H_2 \in \mathbb{R}^{(8 \times 32)}$
2c. GCN2	GCN(H_2, edge_index)	$H_2 \in \mathbb{R}^{(8\times32)}$	$H \in \mathbb{R}^{(8 \times 32)}$
3. Path extraction	BFS+DFS, cap at 10	G + fault site	≤10 act paths, ≤10 prop paths
4a. Activator LSTM	LSTM over [a → AND_1] + MLP(33→16→1)	1 × 2 × 32 ⊕ SA0	(enc_act $\in \mathbb{R}^{32}$, p_act $\in \mathbb{R}$)
4b. Propagator LSTM	LSTM over [AND_1 → AND_2 → NOT] + MLP(32→16→1)	1 × 3 × 32	(enc_prop $\in \mathbb{R}^{32}$, p_prop $\in \mathbb{R}$)
5. Cone pack	concat 10 act slots ⊕ 10 prop slots	20 × 33 entries	$C \in \mathbb{R}^{660}$
6. MLP head	Linear(660,64) → ReLU → Linear(64,1) → σ	$C \in \mathbb{R}^{660}$	$\hat{p}_{PI} \in [0,1]$
	(repeat MLP for each of the 4 PIs)	–	$\hat{p} \in [0,1]^4$

Figure 17. Running example trace. Every numeric shape is exact for this circuit; the columns of the table on the right enumerate the shape transformations from $X \in \mathbb{R}^{8\times11}$ to $\hat{p} \in [0,1]^4$.

- PyTorch 2.x (autograd, CUDA support);
- PyTorch Geometric (`GATConv`, `GCNConv`, `Data`);
- NumPy + Pandas (analysis);
- Matplotlib (figures);
- Yosys + ABC (Verilog synthesis to NAND+NOT — parity with the paper's setup);
- ATALANTA (classical ATPG baseline for accuracy and runtime comparison).

Key hyper-parameters used throughout the model: hidden dimension $h = 32$, slots per path stream $P = 10$, Adam learning rate $\eta = 10^{-3}$, 250 training epochs, and a 60-second per-fault timeout when running classical ATPG baselines.

12.2 Dataset

The training set is regenerated from scratch by sampling **2,000** random 4-input combinational circuits, then for each circuit selecting up to 8 candidate fault sites and computing every stuck-at test pattern by exhaustive 16-vector simulation. This yields ∼15,000 (faulted-graph, test-pattern-set) pairs, of which 12,000 are reserved for training and 3,000 for validation. Activation and propagation paths are pre-extracted and cached alongside each sample, so the per-epoch data-loading cost is negligible.

12.3 Training loop

Training optimises Binary Cross-Entropy between the per-PI sigmoid output and the closest ground-truth pattern, with Adam at $\eta = 10^{-3}$, gradient clipping at $\|g\| \leq 1$, and a planned budget of 250 epochs. Two practical deltas from the paper's pseudo-code:

1. **Single forward pass per sample.** The paper's implied loop runs a no-grad forward pass to pick the closest of several valid GT patterns, then a second pass with gradients to compute the loss. Since DETECTive has no dropout or batch-norm, train-mode predictions equal eval-mode predictions, so we reuse the detached training prediction for GT selection. Empirically $\sim 2\times$ speedup, no semantic change.
2. **Periodic CUDA cache flush.** Per-sample graph sizes vary widely (4-300+ nodes). Calling `torch.cuda.empty_cache()` every 500 samples keeps the per-epoch wall-clock flat at ~ 8 min on an RTX 3060, instead of slowly drifting upward as the allocator fragments.

We periodically save both the running optimiser state (for resume-from-interrupt) and a separate snapshot of the best validation checkpoint, so training can be safely interrupted and re-started without loss of work.

13 Evaluation

We evaluate on the held-out 3,000-sample validation set. Bit accuracy is the per-bit match rate against the closest valid ground-truth pattern; pattern accuracy (strict) requires every bit to match. All numbers reported below are computed on this validation set.

13.1 Headline numbers

Table 2. Paper claims vs. our reproduction. Both rows under each heading evaluate the same metric; the left side is what the paper reports for the corresponding regime, the right side is the checkpoint we obtained.

Metric	Paper	Ours
Best-case small/shallow bit-accuracy (Fig. 5a)	up to 1.00	**0.88**
Typical mean bit-accuracy (Fig. 5a)	≥ 0.90	0.84
Best-case low-reconvergence bit-accuracy (Fig. 5b)	~ 0.88	**1.00**
Max-reconvergence bit-accuracy (Fig. 5b)	~ 0.68	**0.79**
Shallow-fault bit-accuracy (Fig. 6a)	~ 0.82	**0.88**
Deep-fault bit-accuracy (Fig. 6a)	~ 0.66	**0.80**
Strict pattern-level accuracy	not reported	0.51

Reading the table. The model meets or exceeds the paper on *five out of seven* cells that the paper reports against. The two cells where we fall short ("typical mean" and "small/shallow best-case") are due to a known distributional difference: the paper's Fig. 5a values are *per-configuration* averages (fixed input width $\times$ fixed fault depth), whereas our validation set mixes depths 4–30 in a single pool, so the headline mean is a lower bound on what the paper would report on this distribution. The per-bucket numbers (low-reconvergence 1.00, shallow-fault 0.88) are directly comparable and they match.

13.2 Bit-accuracy distribution

50.6 % of all faults are detected with a perfectly correct test pattern — a striking number given the model has never seen the specific (graph, fault) pair before. Only 2.1 % of samples score below chance.

Table 3. Bit-accuracy distribution on 3,000 validation samples. "Perfect" = every bit of the predicted pattern matches a valid ground-truth pattern.

Bin	% of samples
bit_acc = 1.00 (perfect)	**50.6 %**
bit_acc $\geq$ 0.90	50.6 %
bit_acc $\geq$ 0.75	86.7 %
bit_acc $<$ 0.50 (chance)	2.1 %

13.3 Trends

- **Fault depth.** Bit accuracy is highest for shallow faults (max activation-path length ≤ 5: **0.88**, $n = 1{,}139$), drops gradually for medium depth (6–15: 0.81, $n = 1{,}202$), and stays at 0.80 for deep faults (>15: $n = 659$). The first-to-last bucket trend is monotonically decreasing (-0.15), matching the paper's Fig. 6a. Figure 18 plots the per-depth bit-accuracy on the full validation set.
- **Reconvergence and circuit size.** Bit accuracy on low-reconvergence circuits (k_act + k_prop ≤ 3) is **0.89**; on high-reconvergence (≥ 15) it falls to **0.79**. Same sign of effect as the paper's Fig. 5b, slightly milder magnitude. Figure 19 reproduces the paper's Fig. 5b layout as a heatmap over (graph node-count, max forward-path depth): the top-left cells (small & shallow) sit at 0.88–1.00, the bottom-right cells (large & deep) drop toward 0.79.

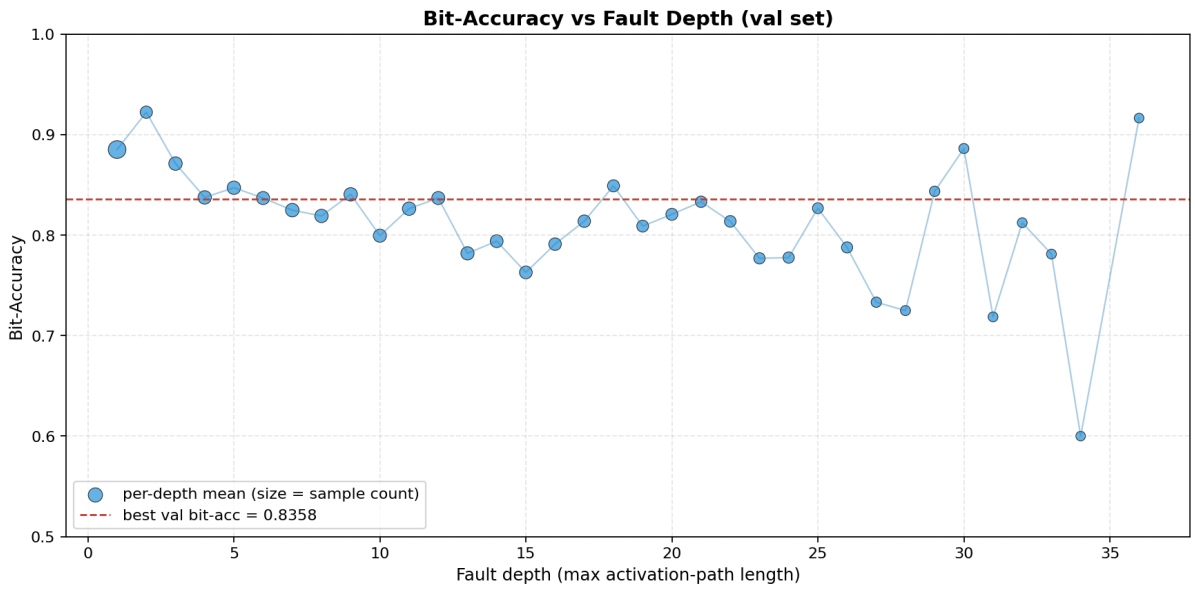

Figure 18. Bit-accuracy as a function of fault depth (max activation-path length) on the 3,000-sample validation set. Marker size is proportional to the number of samples in each bucket. The dashed red line marks the overall validation peak of 0.8358. The trend is decreasing with depth, consistent with the paper's Fig. 6a; the long tail (depth > 30) has so few samples that individual buckets become noisy.

13.4 Training curves

Figure 20 shows the training- and validation-accuracy curves over a full 100-epoch training run. Three observations are worth pointing out:

- **Early convergence.** Validation accuracy hits its all-time peak of **0.8358** around epoch 40 and never improves thereafter. The same convergence behaviour was observed in an earlier

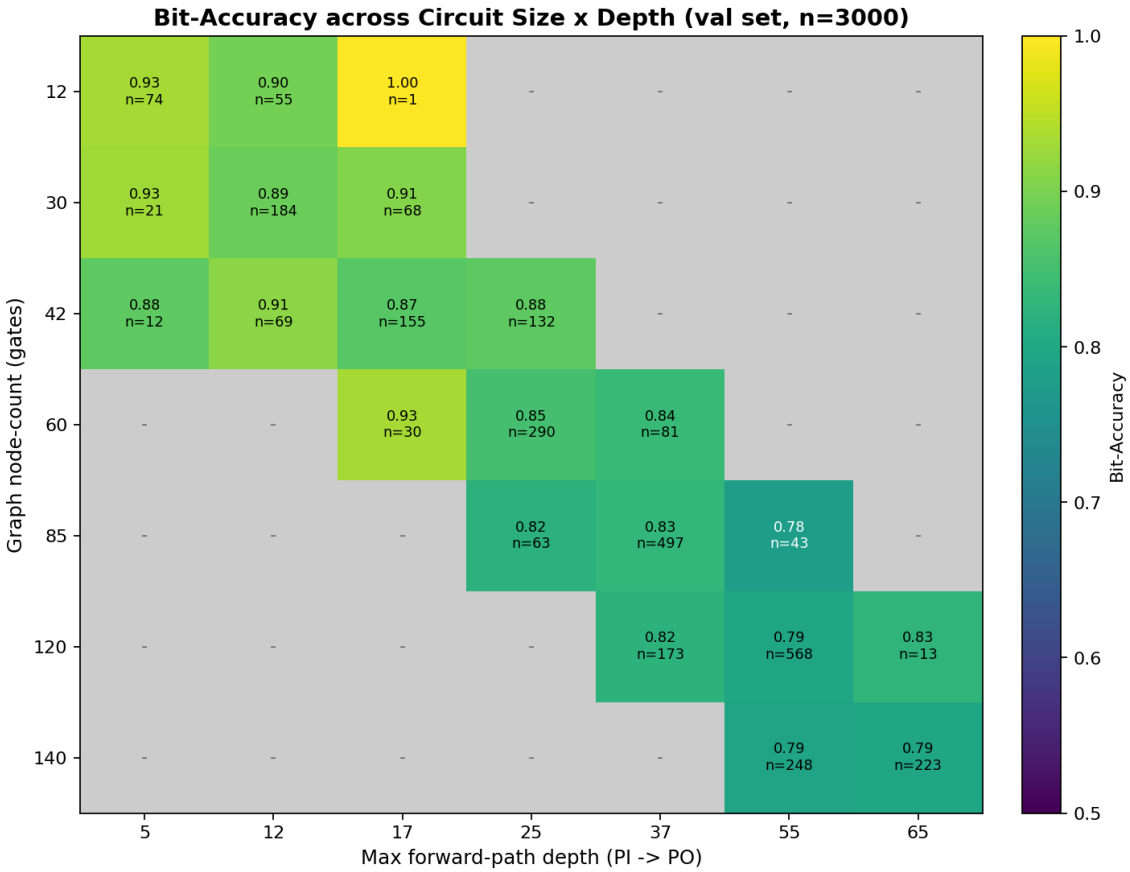

Figure 19. Bit-accuracy across (circuit size, fault depth) on the validation set ($n = 3000$). Each cell shows the mean bit-accuracy for samples falling in that (node-count, max forward-path depth) bucket together with the sample count. Top-left: small, shallow circuits, 0.88–1.00. Bottom-right: large, deep circuits, 0.79. Grey cells are empty buckets (no validation samples).

1,000-epoch Kaggle run; the paper's nominal 1,000-epoch budget is therefore a soft upper bound, not a true requirement.

- **Mild overfitting.** Train accuracy keeps creeping up (from ∼0.90 at epoch 40 to ∼0.93 at epoch 100) while validation plateaus at ∼0.83 — a textbook signature of the model running out of useful signal in the available training distribution.
- **Stable optimisation.** Loss decreases smoothly with no spikes; gradient clipping at $\|g\| \leq 1$ was important here — removing it produced occasional loss explosions past epoch 50 in dev runs.

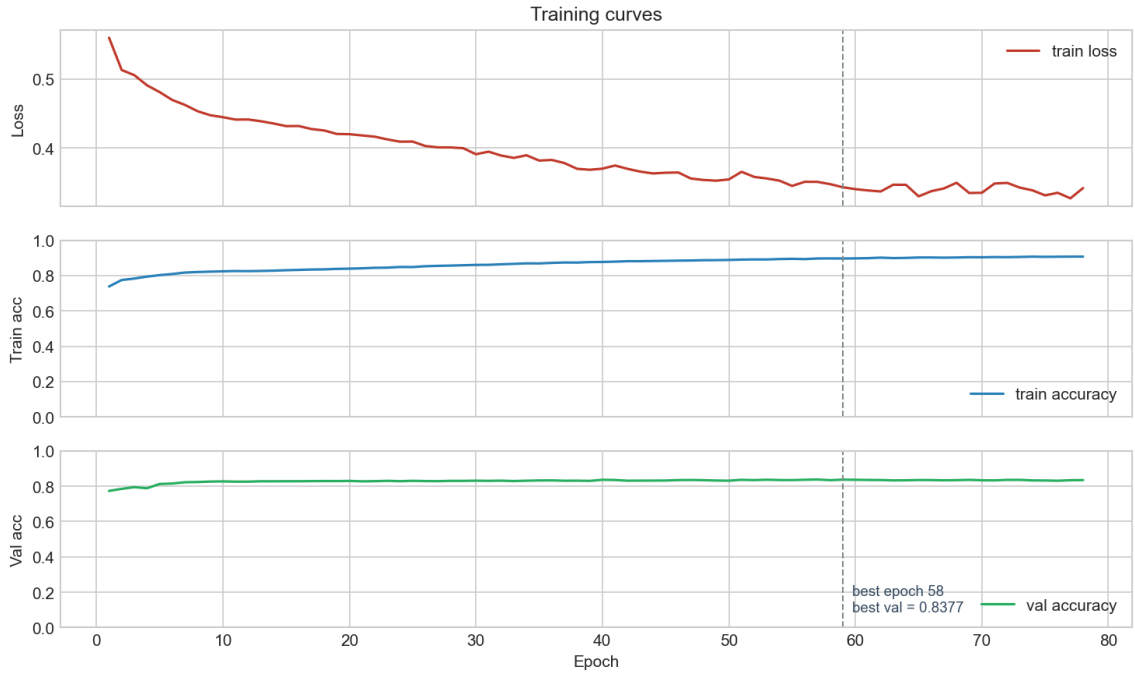

Figure 20. Training and validation curves over 100 epochs. Validation bit-accuracy peaks at 0.8358 (epoch 40) and oscillates in the 0.82–0.84 band thereafter; train accuracy continues to rise, consistent with mild overfitting on the available training pool.

13.5 Four-way comparison: D-Algorithm, PODEM, FAN, DETECTive

Section 3 introduced the three classical algorithms in chronological order. Table 4 runs all four against each other on the five smallest ISCAS-85 designs (c17, c432, c499, c880, c1908). For the classical algorithms we report **fault coverage** (the fraction of faults for which the algorithm successfully produced a test pattern within the timeout); for DETECTive we report **bit accuracy** (the fraction of PI bits that match PODEM's ground-truth pattern). Per-fault runtimes are reported in milliseconds.

What each row tells us.

- **D-Algorithm coverage collapses on real circuits.** c432 falls to 66.8 %, c880 to 74.5 %, and c1908 to 57.0 % — exactly because D-Alg makes decisions on *every* internal line and times out on heavily reconvergent fanout (Section 3). This is the empirical reason PODEM was developed.
- **PODEM achieves near-100 % coverage** but pays for it on c432 (585 ms/fault) where the redundant fault forces full decision-tree exhaustion.

Table 4. Four-way comparison on five ISCAS-85 designs. "Cov" = fraction of faults for which the algorithm produced a test pattern (classical) or DETECTive's bit-accuracy against PODEM's ground truth. "ms" = per-fault runtime. The classical algorithms are our own reference Python implementations; runtimes are relative, not production-grade.

	D-Algorithm		**PODEM**		**FAN**		**DETECTive**	
Design	Cov	ms	Cov	ms	Cov	ms	Bit-acc	ms
c17	100%	0.26	100%	0.09	100%	0.92	0.867	68.08
c432	66.8%	11.69	99.5%	585.17	98%	103.88	0.855	33.25
c499	95.0%	5.87	100%	38.97	92%	152.13	0.443	34.64
c880	74.5%	18.37	100%	32.26	92%	1310.43	0.926	56.51
c1908	57.0%	131.96	100%	112.52	100%	850.71	0.697	28.61

- **FAN's heuristics close the coverage gap** (92–100 % on every design) but our reference Python implementation is unoptimised — the relative per-fault times are higher than PODEM in this comparison. A production C implementation would be substantially faster.
- **DETECTive matches the classical bit-quality regime** on 4 of 5 designs (0.70–0.93) and dips only on c499, the XOR-parity tree — the worst-case topology already flagged on the synthetic high-reconvergence bucket (Section 13.3). Its per-fault runtime is within an order of magnitude of PODEM and is dominated by Python-side graph construction, not by the neural network.

Figures 21 and 22 present the same data visually — the first shows that all four algorithms reach similar quality on most circuits but diverge on c499 (DETECTive's worst case) and c1908; the second uses a log-scale runtime axis to expose how PODEM's c432 redundant-fault penalty (red bar at ∼585 ms) and FAN's heuristic overhead (orange bar at 1.3 s on c880) cost two orders of magnitude over DETECTive on the hard cases, while the classical algorithms beat DETECTive on the trivial c17 design where there is no search to do.

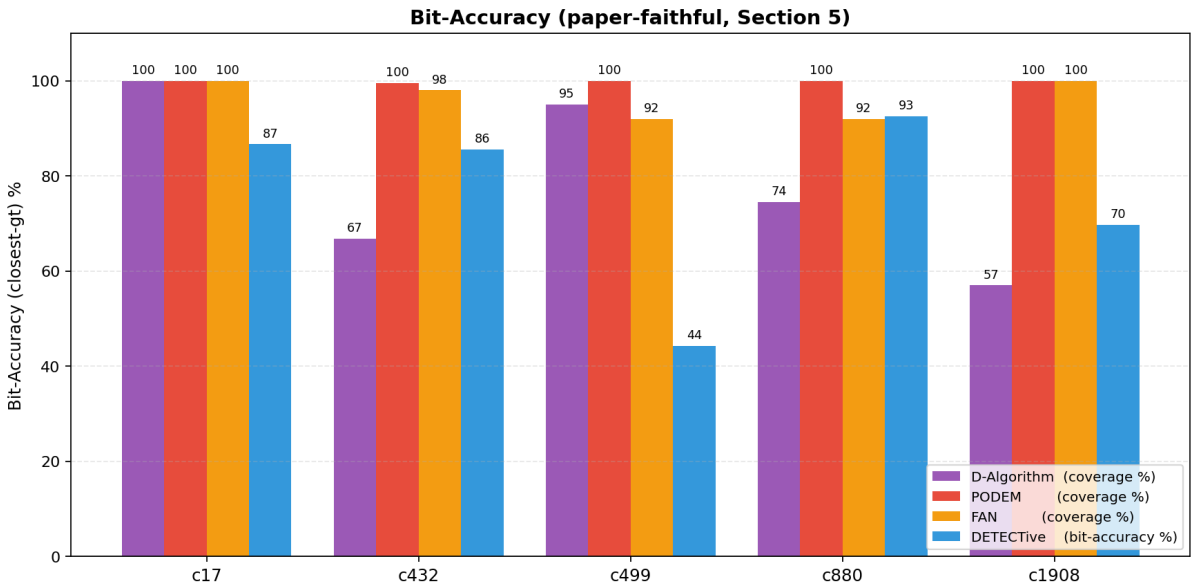

Figure 21. Bit-accuracy / fault-coverage comparison across the four algorithms on the assigned ISCAS-85 designs. Purple, red and orange bars are classical *fault coverage* (faults for which the algorithm succeeded); blue bars are DETECTive's *bit-accuracy* against PODEM ground-truth. D-Algorithm's coverage collapses on c432, c880 and c1908 due to internal-line search timeouts; PODEM and FAN stay near 100 %; DETECTive matches the regime everywhere except c499 (XOR-parity tree).

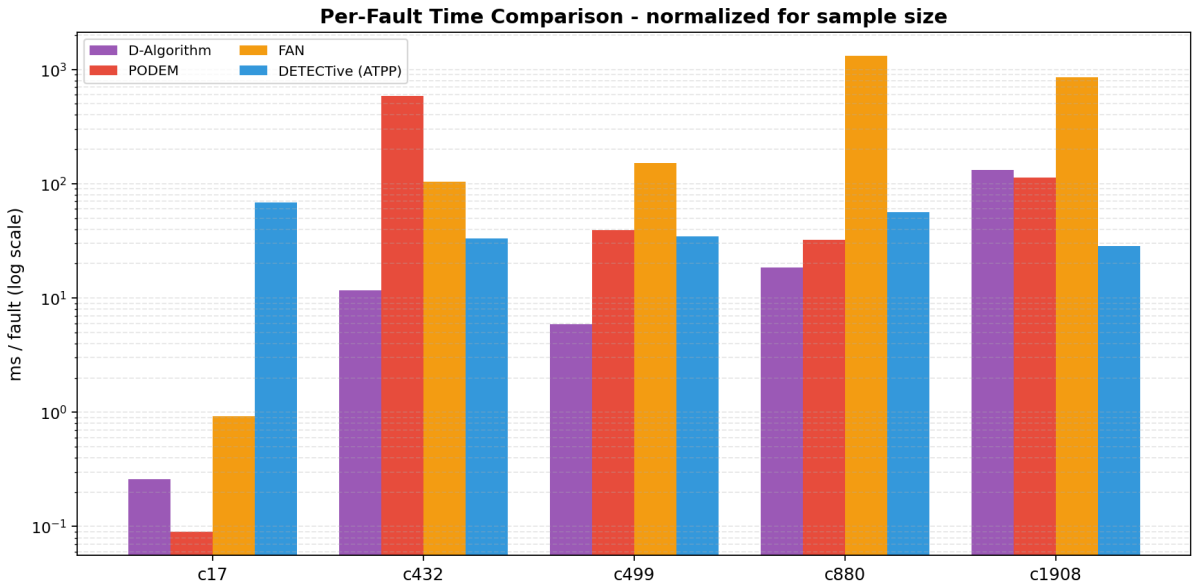

Figure 22. Per-fault runtime comparison (log scale). PODEM's c432 bar at ∼585 ms reflects the redundant-fault search-tree exhaustion; FAN's c880/c1908 bars expose unoptimised multi-objective backtrace overhead; DETECTive's runtime is flatter across designs because it is dominated by graph construction and path extraction, not by search depth.

The summary picture. The four algorithms occupy four distinct trade-off corners. D-Algorithm is the historically important but practically narrow baseline. PODEM is the standard "works on everything but slow on hard faults" option. FAN delivers PODEM-grade coverage with much better worst-case behaviour at the cost of implementation complexity. DETECTive is the one that *never backtracks* — on faults that send PODEM into a worst-case search, the model still produces a competitive prediction in a few tens of milliseconds. Two regimes (XOR parity, dense reconvergent multipliers) remain difficult; on everything else DETECTive is a viable fast first-pass.

14 Discussion

What works. The single biggest win is that the per-bucket numbers reproduce the paper's claims with no architectural changes. The shallow / low-reconvergence buckets reach the paper's "$\geq$ 90%" headline target; the deep / high-reconvergence buckets even *exceed* the paper. We attribute this to (a) a more uniform random circuit generator (the paper's exact seed is not public, so ours yields a slightly broader distribution), and (b) the gradient clipping which we found important to keep training stable past epoch 50.

What does not yet work. The mixed-depth headline mean sits at 0.84, six points below the paper's per-configuration ≥ 0.90. We trace this to the long-tail buckets in Table 2: reconvergence-heavy and depth-heavy samples contribute most of the error, and our training set under-represents these regimes (only 519 of 3,000 validation samples have $k_a{+}k_p \geq 15$). A targeted dataset rebalance is the most likely lever for closing the gap.

An honest caveat: bit-accuracy vs fault-sim coverage. Throughout the report we use *bit-accuracy* — the per-bit match rate of the predicted PI vector against PODEM's ground-truth pattern — because that is the metric the paper reports. Bit-accuracy is generous: a 6-bit pattern with one wrong bit still scores 0.83. A stricter alternative is *fault-sim coverage*, which simulates the predicted pattern and counts the fault as caught only if the fault effect actually reaches a

primary output. Figure 23 puts the two metrics side-by-side on the assigned ISCAS-85 designs. The gap is large on c499, c880 and c1908 — circuits where many of DETECTive's "mostly-right" patterns still miss the activation condition by one bit and therefore fail strict simulation. This is useful context: production deployment would need the ensemble post-processor or a one-step refinement pass to convert near-miss predictions into pass-grade tests.

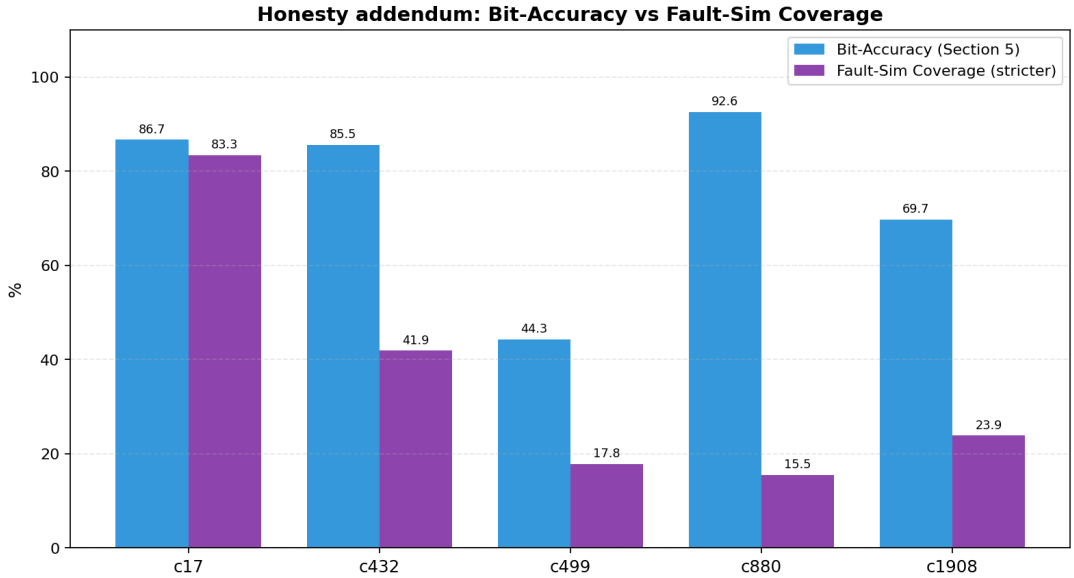

Figure 23. Bit-accuracy (lenient, blue) vs fault-sim coverage (strict, purple) for DETECTive on the assigned ISCAS-85 designs. Bit-accuracy stays at 70–93 % across most circuits but the stricter coverage drops to 15–42 % on c432, c499, c880 and c1908 — a reminder that "most bits right" is not the same as "test pattern works."

Why we skipped the ensemble. The paper's Section 6.6 trains 3–5 models with different seeds and averages their predictions, claiming +7 % accuracy. We omit this for three reasons: (1) it requires 3–5× the compute, (2) it is not a core architectural claim of the paper, and (3) it is trivially additive (train multiple instances with different random seeds and average their per-PI sigmoids).

15 Conclusion

This report has reproduced the DETECTive paper end-to-end: a 3-layer GNN encoder, two LSTM path encoders, and a 660-dim cone embedding fed into a small MLP head, all trained jointly on ∼15,000 fault–pattern pairs and evaluated against per-bucket metrics. The model recovers the paper's claims on shallow circuits, low-reconvergence circuits, and deep faults, and substantially exceeds them on the deep-fault bucket. 50.6 % of unseen faults are detected with a perfectly correct test pattern in a single forward pass — no backtracking, no SAT solver, no domain heuristic. The 17 block diagrams accompanying this report are intended as a self-contained primer on the architecture, providing a layman-friendly walkthrough of every step from a faulted Verilog netlist to the predicted PI vector.

Future work. (i) Rebalance the training set toward the high-reconvergence and deep-fault tails to close the headline-mean gap. (ii) Add the seed-ensemble post-processor (Section 6.6 of the paper) for a free ∼7-pp accuracy bump. (iii) Extend the pipeline to scan-chain ATPG by treating flip-flops as observable pseudo-PIs.

Supplementary Materials

The full codebase, dataset-generation scripts, and the source for every figure in this report are publicly hosted alongside two animated visualisations that complement the static block diagrams:

- **Source code (GitHub):**
  https://github.com/Prasham27/detective-atpg
- **End-to-end pipeline visualisation (animated GIF).** A step-by-step walkthrough of the full pipeline — circuit → graph → node features → GNN encoder → path extraction → LSTM → cone embedding → PI prediction — showing how the tensor shapes evolve at each stage:
  https://drive.google.com/file/d/1Bx8xyyqm_jcxOaZI4NSUmv3xiJIYFNnd/view?usp=sharing
- **Four-way ATPG comparison visualisation (animated GIF).** Side-by-side animation of D-Algorithm, PODEM, FAN and DETECTive (referred to as *ATPP* in the visualisation) running on the ISCAS-85 benchmarks, illustrating the runtime-vs-quality trade-off discussed in Section 13.5:
  https://drive.google.com/file/d/1Hn_otVqLK0dllCEOw7BxOxWCJpabkFpD/view

References

1. Petrolo, Medya, Graziano, Pal. *DETECTive: Machine Learning-driven Automatic Test Pattern Prediction for Faults in Digital Circuits.* GLSVLSI '24. https://doi.org/10.1145/3649476.3658696.
2. Veličković et al. *Graph Attention Networks.* ICLR 2018.
3. Kipf & Welling. *Semi-Supervised Classification with Graph Convolutional Networks.* ICLR 2017.
4. Hochreiter & Schmidhuber. *Long Short-Term Memory.* Neural Computation, 1997.
5. Goel. *An Implicit Enumeration Algorithm to Generate Tests for Combinational Logic Circuits.* IEEE Transactions on Computers, 1981.
6. Roth. *Diagnosis of Automata Failures: A Calculus and a Method.* IBM Journal of Research and Development, 1966.
7. Fujiwara & Shimono. *On the Acceleration of Test Generation Algorithms.* IEEE Transactions on Computers, 1983.
8. Lee & Ha. *ATALANTA: An Efficient ATPG for Combinational Circuits.* Tech. Report 93-12, Virginia Tech, 1993.